

FULL-STACK PROJECT TASK BRIEF

Employee Management System

Build a production-ready HR portal with authentication, advanced CRUD, search, pagination, image upload & more.

Next.js	Node.js	Express.js	MongoDB	JWT Auth
---------	---------	------------	---------	----------

12 Core Feature Areas · Evaluation Checklist Included · Beginner Friendly

OBJECTIVE

Build a full-stack **Employee Management System** using Next.js (Frontend), Node.js, Express.js, and MongoDB — demonstrating practical skills in REST API design, database modelling, JWT authentication, file uploads, form validation, and responsive UI.

1 Authentication

- User registration
- User login
- JWT authentication
- Password hashing
- Logout
- Protected routes

2 Dashboard

- Display total employees
- Active employees count
- Inactive employees count
- Department count
- Recent employees

3 Employee CRUD

- Create employee
- View employee list
- View single employee details
- Update employee
- Delete employee
- Soft delete support
- Restore deleted employee

4 Search & Filtering

- Search by name
- Search by email
- Filter by department
- Filter by status
- Filter by joining date
- Multi-filter support

5 Pagination & Sorting

- Server-side pagination
- Sort by name
- Sort by salary
- Sort by joining date
- Ascending/descending order

6 Form Validation

- Required field validation
- Email validation
- Phone number validation
- Duplicate email prevention
- Error messages
- Client-side and server-side validation

7 Employee Fields

Each employee record must include the following fields:

Field	Notes
Full Name	Primary identifier for the employee
Email	Unique — used for login and notifications
Phone Number	Validated format, required field
Department	Used for filtering and department count
Position	Job title within the department
Salary	Numeric value, sortable
Date of Joining	Used for filtering and sorting
Address	Optional text field
Profile Image	Uploaded file, stored as URL
Status	Active or Inactive — controls dashboard counts

8 Image Upload

- Upload profile image
- Preview before upload
- Store image URL
- Update image
- Delete image

9 API Features

- RESTful API
- Proper HTTP status codes
- Error handling
- Validation middleware
- Centralized error handler

10 Database

- Mongoose models
- Schema validation
- Timestamps
- Indexing for search
- Relationships where needed

11 UI Requirements

- Fully responsive
- Loading spinner
- Skeleton loading
- Empty state UI
- Confirmation modal before delete
- Toast notifications
- Clean dashboard design

12 Bonus Features

Bonus Feature	Description
Role-based access	Admin / User roles with protected endpoints
Dark mode	Toggle via Tailwind dark class
Export to CSV	Download full employee list as .csv
Import from CSV	Bulk-upload employees via CSV file
Activity logs	Track create / update / delete actions
Audit trail	Record who changed what and when
Profile page	Logged-in user can view/edit their profile
Change password	Secure password update with old-pass verify
Forgot password	Email-based password reset flow
Reset password	Token-validated new password form
Email verification	Confirm email on register before login

EVALUATION CHECKLIST

- Clean folder structure
- Reusable components
- Proper API architecture

- Secure authentication
- Good error handling
- Clean code and naming conventions
- Responsive UI
- Git commits with meaningful messages
- Well-documented README
- No unnecessary code duplication

This task is substantial enough to demonstrate practical skills in **Next.js, Express.js, MongoDB, authentication, API design, and advanced CRUD operations**. Work through each feature area in order, commit your code regularly, and document your setup clearly in the README.